Detecting and escorting a drone from another drone: one small detector, a cheap tracker and strict test discipline

Javier Chaos · November 2026
Personal side project ("Drone Volador"). Licence: CC BY 4.0.
linkedin.com/in/javierchaos · github.com/jachaos

Abstract. We describe how we built the vision system of an escort drone: a quadcopter that must find another drone (any brand, no markers, no cooperation) as early as possible with its own camera and then follow it without ever switching to the wrong target. The system is one single-class YOLO26-s detector (9.5 M parameters) used at two input sizes, plus a region-of-interest tracker that costs about one millisecond per frame. The detector was trained for 80 epochs from the COCO checkpoint on 34,854 images from five public sets and our own footage. On a hand-reviewed test set of four videos never used for training (2,169 drone instances) it sees 93.3 % of the drones with 3.6 false boxes per 100 frames at about 4 ms per 1280×800 frame on a desktop GPU. We report the full evaluation protocol, the model history including fourteen approaches that did not work, a comparison of five frame-to-frame trackers, and a step-by-step recipe to reproduce the detector from public data.

Contents

1. Problem and constraints 2

2. System overview 3

2.1 Search and the zoomed re-check 3

2.2 Identification: whom to escort 4

2.3 Follow: what happens in every frame 4

2.4 Confirm: the detector in the loop 8

2.5 Losing the target, and finding it again 8

2.6 Frame-to-frame trackers we compared 9

2.7 Multi-drone variant 11

3. The detector: data and training 12

3.1 Training data 12

3.2 Training recipe 14

3.3 Deployment 15

4. Evaluation protocol 15

5. Results 16

5.1 Model history 16

5.2 Escort results 18

5.3 Why 80 epochs 18

5.4 What the detector sees and misses 20

6. What worked and what did not 21

7. Limitations and open problems 22

8. Reproducing the detector 23

9. Hardware, compute and the simulator 24

9.1 Hardware and software 24

9.2 Compute used 24

9.3 The simulator 25

References 26

1. Problem and constraints

Our drone escorts other drones that perform tasks in our area. It has no GPS and must rely on its camera to find the target and keep it at 15 m or less while the target manoeuvres. The target is any multirotor: we cannot add LEDs or markers to it, and several similar drones may be in view at once (races, swarms). Operation is outdoors, in daylight, without fog, rain or strong wind.

Four constraints shaped every decision below.

- **The camera.** A colour global-shutter sensor at **1280×800** (OmniVision OV9782) with a 60° lens. Everything is trained and tested at 1280 px wide; a drone at 50 m is 10 to 20 pixels. Part of our footage was recorded with the monochrome version of the same sensor (OV9281), which is why every training image also exists in grayscale.
- **Compute.** The vision chain must stay well under 30 ms per frame on a desktop GPU now and run on a Raspberry Pi 5 with a Hailo accelerator later. This rules out large transformers and anything that runs the detector on every frame at full resolution.
- **Never jump to another drone.** Locking onto the wrong drone is the worst failure, worse than losing the target for a second. Every rule in the tracker is written with this in mind: *better lost than wrong*.

- **Detection first.** The tracker is cheap and deliberately simple (no Kalman filter, no learned motion model). Whenever it is in doubt, it asks the detector. Detector quality therefore decides both how early we acquire and how quickly we recover, and we measure the detector in two ways: alone, and inside the escort loop.

2. System overview

The whole system is one detector and one tracker (Figure 1). The detector is a YOLO26-s with a single class, “drone”, exported to TensorRT FP16 at two input sizes: 800×1280 for the whole frame and 640×640 for crops. The tracker works on a small region of interest around the last known position and costs 0.1 to 1.2 ms per frame. A confirmation thread runs the detector on a 640 px crop around the track every 0.1 s and either confirms, corrects or drops the lock.

2.1 Search and the zoomed re-check

In search mode the detector runs on the whole 1280×800 frame at a low threshold (0.25). Boxes with confidence ≥ 0.60 are accepted directly. Boxes between 0.25 and 0.60 get a second look: we crop a square six times the box (at least 256 px) from the full-resolution frame, resize it to 640 px and run the *same* model again; the box is kept if the crop detection is ≥ 0.30 (Figure 2). Small, distant drones that the full-frame pass sees only faintly survive this test; textured background that looked drone-like at 1280 usually does not. On our clean exam the re-check reduces false boxes from 11.1 to 3.6 per 100 frames while giving up 1.7 points of recall (95.0 % → 93.3 %).

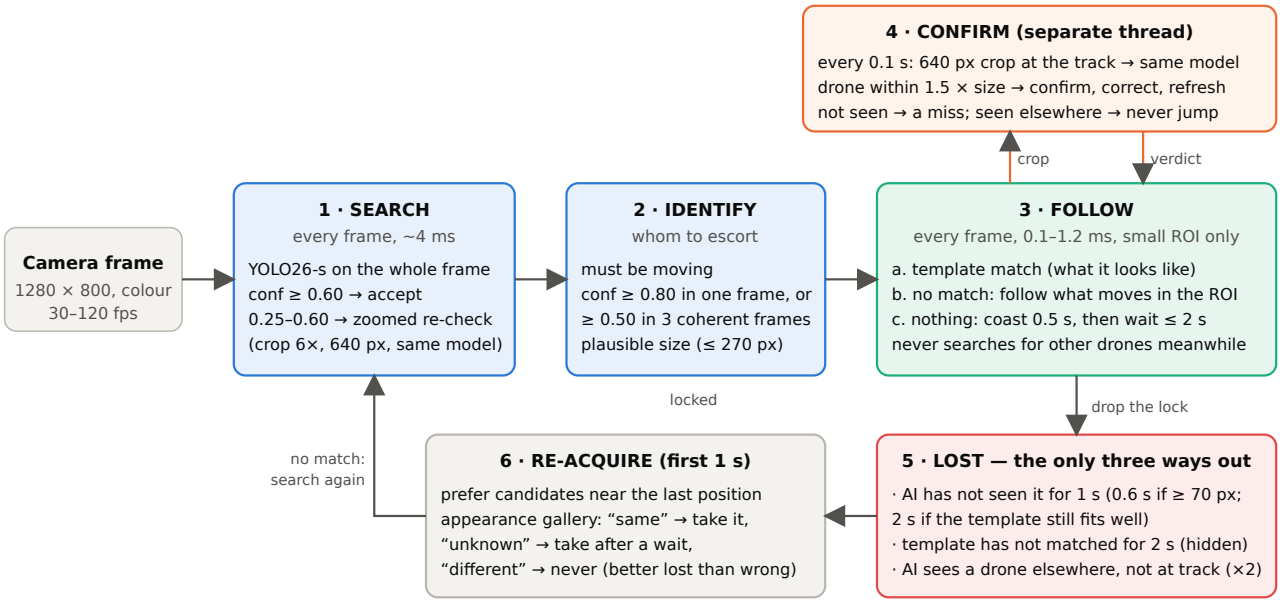


Figure 1. The escort vision loop. One set of YOLO26-s weights serves search (1), identification (2) and confirmation (4); the per-frame work in follow mode (3) is classical image processing on a small crop.

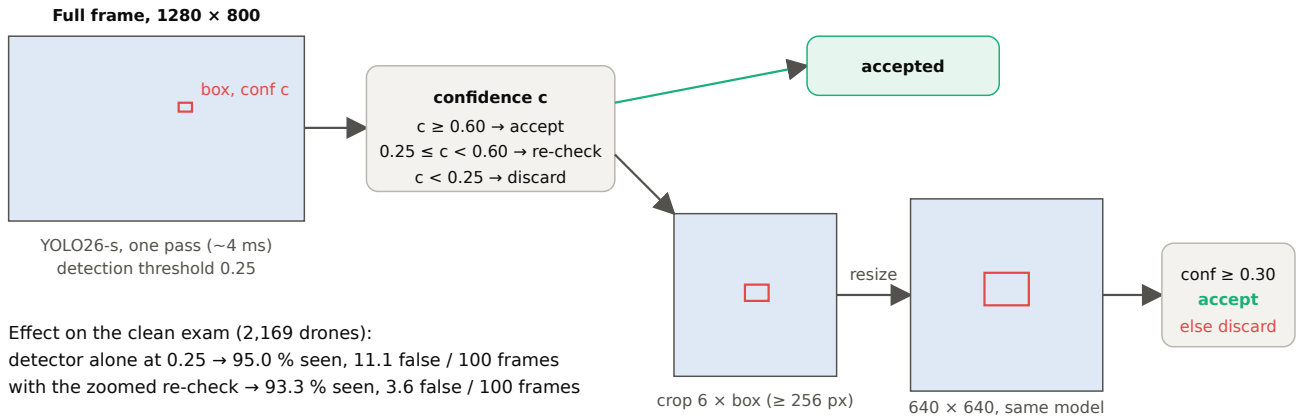


Figure 2. The zoomed re-check. Medium-confidence detections are re-examined by the same network on an upscaled crop; no second model is needed.

2.2 Identification: whom to escort

A detection becomes the escort target only if it is moving (a parked drone is not a target, although a hovering one is followed once locked), if it is not implausibly large, and if the detector is sure: one frame at ≥ 0.80, or three consecutive coherent detections at ≥ 0.50. While a target is being followed, the search for other drones stops.

2.3 Follow: what happens in every frame

Once a target is locked, three clocks run at once (Figure 4). The **frame clock**: every frame, about one millisecond of classical image processing inside a small region around the predicted position. The **confirmation clock**: every 0.1 s a 640×640 crop around the track goes to the detector in a background thread, so the tracker never waits for it. And a handful of **patience clocks** that decide when to give up (1 s, 2 s, 10 s). Every rule below is written so that, in doubt, the lock is dropped rather than moved onto something that might not be our drone.

Figure 3 shows the order of operations in one follow-mode frame; the thresholds are collected in Table 1, so that the text can stay readable.

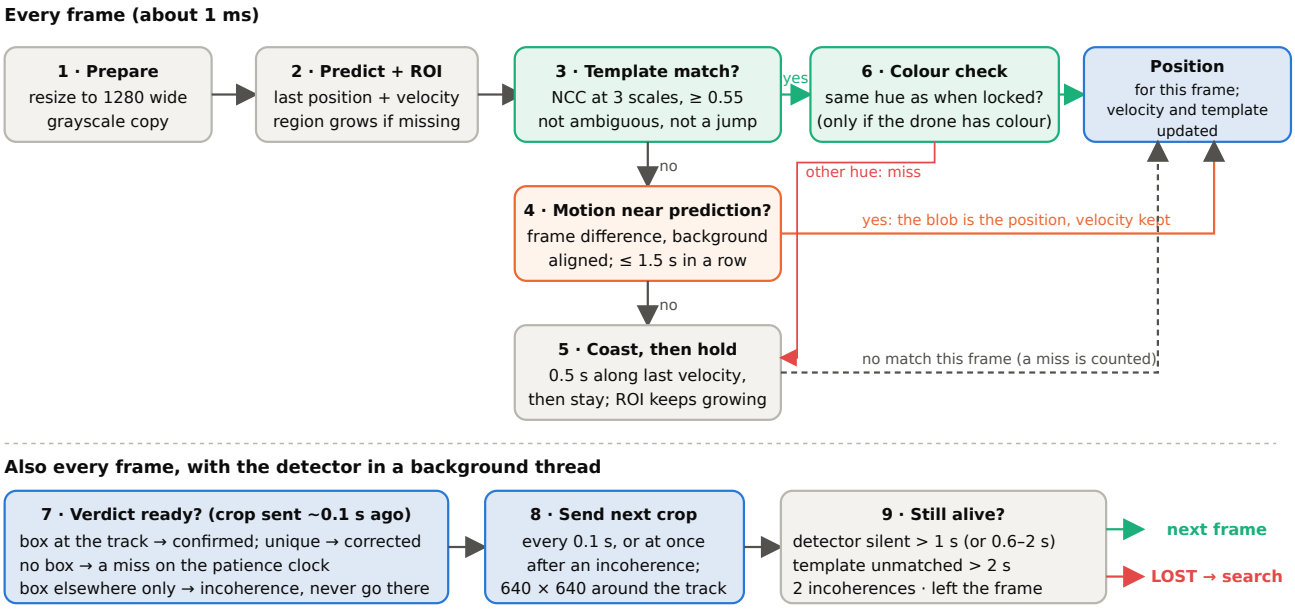


Figure 3. One follow-mode frame, in order. The top row runs every frame and costs about a millisecond; the bottom row reads the detector's verdict when one is ready, sends the next crop every 0.1 s, and decides whether the lock is still alive.

1. Prepare. The frame is resized to 1280 px wide (the camera's own width) and a grayscale copy is made. All template work uses the grayscale copy; the colour frame serves the colour check and the detector.

2. Predict. Where should the drone be now? At the last position plus the displacement measured between the last two good matches. The search region (ROI) is a square around that point whose half-side is the drone's size plus twice its displacement plus 8 px, and never less than 24 px. Every frame without a match the half-side grows by twice the drone's speed plus a quarter of its size, up to 200 px, so a drone that reappears a little off the prediction is still inside the region.

3. Template match. The stored template is a grayscale patch of the drone kept at a scale where the drone measures at most 32 px: a 100 px drone is compared shrunk, which costs the same as a distant one and tolerates its turns better. It is slid over the ROI with normalised cross-correlation at three scales (0.94, 1.00, 1.06, to follow approach and recession; only 1.00 for drones under 20 px, which have too few pixels to resize), with a 0.01 penalty for changing scale so that the size does not jitter. The best peak is accepted only if all of the following hold:

- its correlation is ≥ 0.55, or ≥ 0.40 when it lies within 0.3 template-widths of the predicted spot and the drone was matched in the last three frames (a drone changes aspect as it turns, and we would rather keep it than lose it);
- it is not ambiguous: if a second peak reaches 92 % of the best and lies closer to the prediction than the best does, the tracker switches to it only when it sits within half a template of the predicted spot; otherwise it does not move at all (ambiguity means hold, not guess);
- it is not a jump: a position more than max(½ size, 3 × speed + 8 px) away from the prediction needs correlation ≥ 0.85;
- after more than three missed frames (a reappearance) the bar is higher: correlation ≥ 0.70 and ≥ 0.50 similarity with the anchor;

- the patch at the new position still resembles the **anchor**, the last crop the detector confirmed (correlation ≥ 0.40). This is what stops the template from drifting slowly onto the background.

The peak is then refined to sub-pixel precision with a parabola on each axis. The velocity is measured from the last *good* position divided by the frames elapsed since it (VEL_FIX), and a smoothed copy (60 % old, 40 % new) is kept for coasting. If the correlation is ≥ 0.75 the template is blended 15 % towards the new patch (30 % for drones larger than 64 px, which change aspect faster), so it follows gradual changes without ever being replaced by something the detector has not seen.

4. Motion fallback. If the template fails, the tracker looks for *what moves* near the predicted spot. The previous frame is first aligned to the current one by phase correlation, so that a panning camera does not count as motion; the absolute difference is thresholded at max(14 grey levels, 1.5 × its 95th percentile); connected blobs with an area between 3 % and 400 % of the drone's area are candidates; and the one nearest the prediction is taken if it lies within max(¾ size, ½ speed + ½ size, 8 px) (MOTION_STRICT). Motion never changes the velocity estimate, and it is allowed for at most 1.5 s in a row.

5. Coast, then hold. If neither matches, the position advances along the smoothed velocity for up to 0.5 s (the drone passes behind a tree or a pillar and comes out on the other side), then stays where it is while the ROI keeps growing.

6. Colour check. If the drone had colour when it was locked (at least 15 % saturated pixels in the centre of its box) and the template matched on a patch that also has colour but of a different hue (Bhattacharyya distance between the two hue histograms > 0.5), the match is rejected and counted as a miss. Two race drones crossing are identical in grayscale and differ only in their LEDs; this rule is what kept the tracker from swapping them.

7. Read the detector's verdict. If the confirmation thread has finished the crop sent a few frames ago, its boxes are compared with where the track *was in that frame*. A box within the gate (1.5 × the drone's size, at least 12 px, or 0.8 × the ROI half-side if the tracker was already failing then) **confirms** the lock. If it is the only candidate (the second is more than 1.5 times farther away), the tracker is **corrected**: the template is re-cut from the detector's box in *that* frame, the current position is shifted by the error measured then, and the new template is matched in the current frame; a sample goes to the appearance gallery and the hue reference is updated (70 % old, 30 % new). When the gallery already knows the drone, a box it judges *same* that lies elsewhere wins over a nearer box, and a box it judges *different* can never correct the track. If the detector saw a drone in the crop but none at the track, an **incoherence** is counted; the next confirmation resets the count.

8. Send the next crop. If 0.1 s has passed since the last one, or an incoherence is pending, a 640×640 crop centred on the current position is handed to the detector thread. At 1280 px wide the crop shows the drone at native resolution, with half the frame's width of context.

9. Decide whether the lock is still alive. Four reasons end it (Section 2.5). Otherwise the position is logged and the next frame begins.

Table 1. The follow-mode rules in one place. Sizes are in pixels at 1280 px frame width; "speed" is the smoothed displacement per frame.

Rule	Value in use	Why
Template scale	drone shrunk to ≤ 32 px	Same cost at any distance; a shrunk template tolerates turns
Scales tried per frame	0.94 / 1.00 / 1.06 (only 1.00 below 20 px); 0.01 penalty for changing	Follows approach and recession without size jitter
Accept a match	correlation ≥ 0.55	
Accept a weak match	≥ 0.40 if within 0.3 template-widths of the prediction and matched ≤ 3 frames ago	Aspect changes when the drone turns
Far from the prediction	≥ 0.85 beyond max(½ size, 3 × speed + 8 px)	A jump needs proof
Two similar peaks	second ≥ 0.92 × best: take the closer one only if within ½ template of the prediction, else hold	Never guess between look-alikes
Anchor	≥ 0.40 similarity with the last detector-confirmed crop (0.50 after a gap)	No slow drift onto the background
Reappearance	after > 3 missed frames: correlation ≥ 0.70	Re-locking needs more evidence than keeping
Template upkeep	replaced by the detector box at every confirmation; blended 15 % (30 % above 64 px) when correlation ≥ 0.75	Follow gradual aspect change, never drift
Velocity	from the last good position (VEL_FIX); smoothed 0.6/0.4 for coasting	Misses do not corrupt the speed estimate
Search region	half-side = size + 2 × speed + 8 px (≥ 24); + 2 × speed + ¼ size + 2 px per missed frame, max 200 px	Grows with what the drone could have done
Motion fallback	diff ≥ max(14, 1.5 × p95) after background alignment; blob area 3–400 % of the drone; within max(¾ size, ½ speed + ½ size, 8 px); ≤ 1.5 s in a row; velocity untouched	What moves where the drone should be is probably the drone
Coast	0.5 s along the smoothed velocity, then hold	Passing behind something
Colour check	reject a match if both patches have ≥ 15 % saturated pixels and their hue distance is > 0.5	Race drones differ only by their LEDs
Confirmation	640 px crop every 0.1 s; gate max(1.5 × size, 12 px); correct only if the candidate is unique (second > 1.5 × farther)	Detector as referee, not as tracker
Patience	1 s without confirmation (0.6 s for drones ≥ 70 px; 2 s if the template still matches at ≥ 0.75); 2 s without a template match; 2 incoherences (5 if the template is matching well); out of frame	Better lost than wrong
Re-acquisition	1 s nearest-first; gallery verdict *same* / *unknown* / *different*; gallery cleared after 10 s	Never switch to another drone

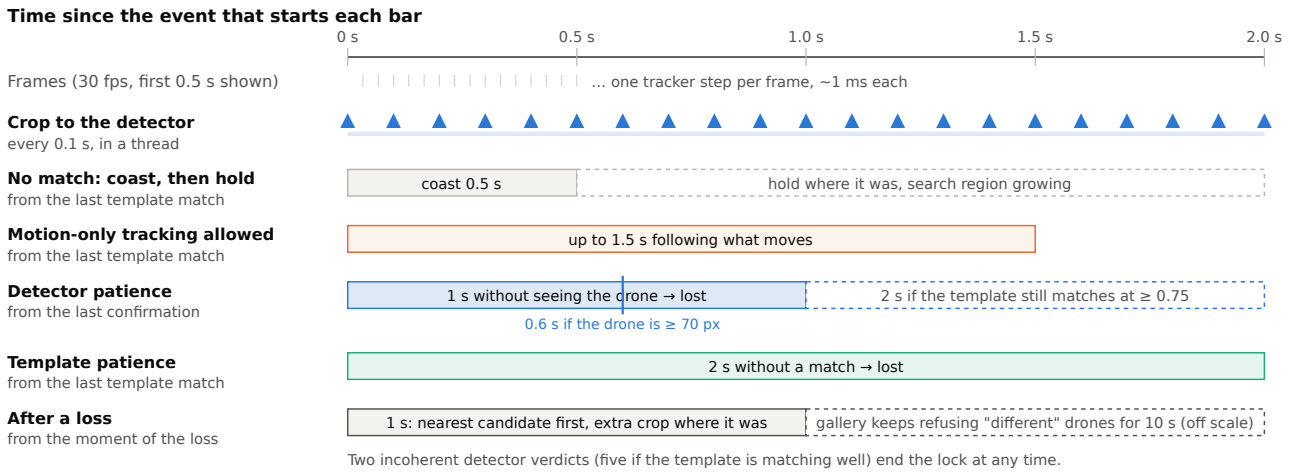


Figure 4. The clocks of follow mode at 30 fps. Each bar starts at its own event (last confirmation, last template match, loss): coasting stops after 0.5 s, motion-only tracking after 1.5 s, the detector is given 1 s to see the drone again (0.6 s when it is large, 2 s when the template is still matching well), the template 2 s, and after a loss the nearest candidate is preferred for 1 s while the gallery remembers the drone for 10 s.

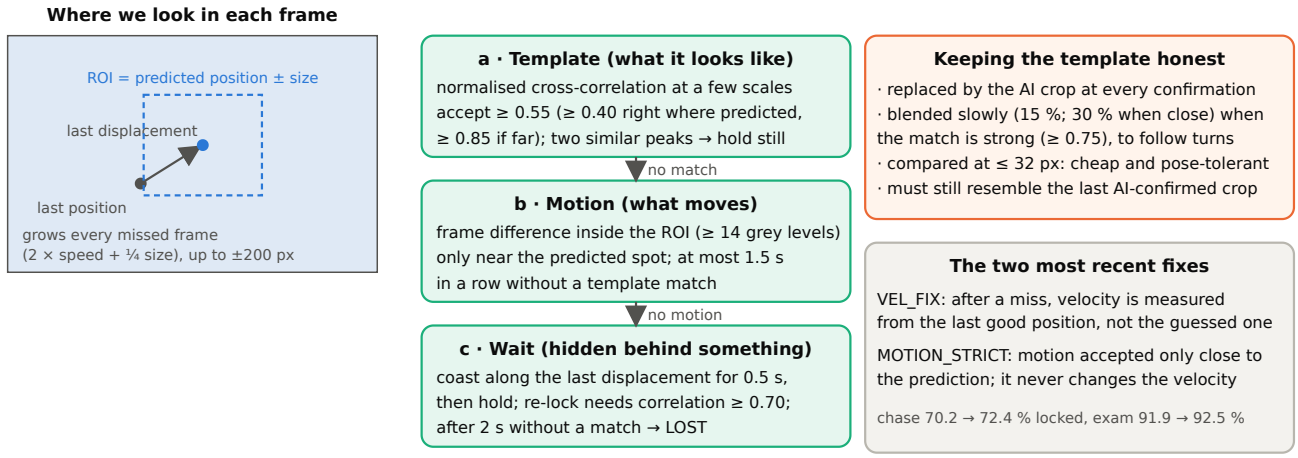


Figure 5. The three per-frame methods and how the template is kept honest. The two fixes on the right (VEL_FIX, MOTION_STRICT) are the most recent additions.

2.4 Confirm: the detector in the loop

The detector is the referee of the tracker, not a second tracker. It runs on a 640×640 crop centred on the track every 0.1 s in its own thread, so a slow answer never stalls the frame loop; its verdict is applied to the frame it was computed on, and the tracker carries the correction forward (step 7 above). Three verdicts are possible: **confirmed** (a box at the track; the position and template are refreshed), **nothing** (a miss is counted against the patience clock), and **elsewhere** (a drone in the crop but not at the track: an incoherence). The tracker never moves to the other drone; after two incoherences it drops the lock and lets the search decide, with the appearance gallery as a guard.

2.5 Losing the target, and finding it again

There are exactly four ways to lose the lock, checked at the end of every frame:

1. the detector has not confirmed the drone for **1 s** (0.6 s for a large drone of ≥ 70 px, which the detector sees almost always; 2 s if the template still matches at ≥ 0.75, because then the tracker itself is trustworthy);

2. the template has not matched for **2 s** (hidden, or gone);
3. the detector has reported the drone **elsewhere twice** (five times if the template is matching well: in a race, other drones in the crop are normal);
4. the predicted position has left the frame.

The patience values were tuned on our videos: 1 s reacts twice as fast as the earlier 2 s without losing more targets. On a loss the frame number, position and size are recorded and the loop returns to search mode.

In search mode every frame runs the full-frame detector with the zoomed re-check (Section 2.1). For **1 s after a loss**, it also runs the 640 model on a crop around the last position (four times the lost drone's size, at least 192 px, upscaled to 640), because a medium-sized drone is seen better magnified than in the whole frame reduced; and candidates are sorted by distance to the last position rather than by confidence, since the most confident box may be another drone. Candidates larger than about 270 px are discarded as implausible. A candidate becomes the target when the detector is sure: one frame at ≥ 0.80, or three consecutive frames at ≥ 0.50 within max(20 px, 2 sizes) of each other. On lock, the template, the anchor, the hue reference and the first gallery sample are taken from the detector's box.

Candidates are also compared with an **appearance gallery** of the lost drone: up to ten grayscale samples at real resolution plus a hue histogram of its lights, collected only while the detector was confirming it. The verdict is *same* (shape and colour both agree), *different*, or *unknown* (tiny drone, no colour, touching the border, empty gallery). *Same* candidates come first; *unknown* ones are taken when no *same* is present (the waiting time before accepting an *unknown* is a parameter, currently zero); *different* ones are never taken, until our drone has been gone for 10 s, when the gallery is cleared and a new target may be chosen. Shape alone is never enough for *same*: two drones of the same model look identical.

Measured on nine of our videos (21,610 frames with a drone), the identity rules removed most jumps to other drones and cut lost-target events by two thirds, at the cost of a few more frames without a position.

2.6 Frame-to-frame trackers we compared

The template tracker is not the strongest tracker on any single video; it is the one that behaves best across all of them. We ran the same escort loop with the tracker replaced by alternatives, keeping the detector (cv15), the confirmation thread and all other rules identical. The confirmation thread corrects each tracker the same way (the alternative trackers are re-initialised with the detector box). Table 2 gives the accuracy, Table 3 the cost, and Figure 6 both at a glance.

Table 2. Accuracy of frame-to-frame trackers inside the same escort loop, detector cv15, colour 1280. "Locked" = frames with the drone where the track is on it; "wrong" = on something else; "changes" = locks that moved to a different drone (must be zero). The chase video is the hardest sequence (FPV chase through a building, Section 4).

Tracker	Chase video: locked / wrong / changes	Nine own videos (21,610 fr): locked / wrong / changes	YouTube set (32,623 fr): locked / wrong / changes
Template + motion + wait (ours, default)	70.2 % / 5.7 % / 0	**86.9 % / 2.7 % / 0**	87.0 % / 5.2 % / 2
ViTTrack (lightweight transformer, OpenCV Zoo)	80.0 % / 3.9 % / 0	73.0 % / 2.1 % / 2	87.2 % / 4.4 % / 2

Tracker	Chase video: locked / wrong / changes	Nine own videos (21,610 fr): locked / wrong / changes	YouTube set (32,623 fr): locked / wrong / changes
NanoTrack v2 (siamese network)	81.1 % / 6.4 % / 0	83.8 % / 7.6 % / 3	87.5 % / 5.8 % / 3
CSRT (discriminative correlation filter)	79.8 % / 5.7 % / 0	not run (slowest)	not run
Detector on the crop every frame	80.2 % / 3.9 % / 0	89.0 % / 2.7 % / 2	87.2 % / 5.0 % / 3
Hybrid: template below 40 px, NanoTrack above	78.5 % / 5.6 % / 0	not run	not run
NanoTrack + detector every frame	79.5 % / 5.9 % / 0	79.5 % / 6.8 % / 3	85.5 % / 6.0 % / 3

Table 3. Cost of the same trackers: median milliseconds per frame for the tracker alone (the detector's confirmation thread is the same for all and is not included). CPU unless stated; desktop computer.

Tracker	ms per frame	Relative to the template
Template + motion + wait (ours)	**1.1** (95th percentile 4.2)	1×
ViTTrack	4.2	3.8×
NanoTrack v2	5.5	5×
Hybrid (template / NanoTrack)	7.7	7×
Detector on the crop every frame	7.9 (GPU)	7×
CSRT	10.1	9×
NanoTrack + detector every frame	30.7 (GPU + CPU)	28×

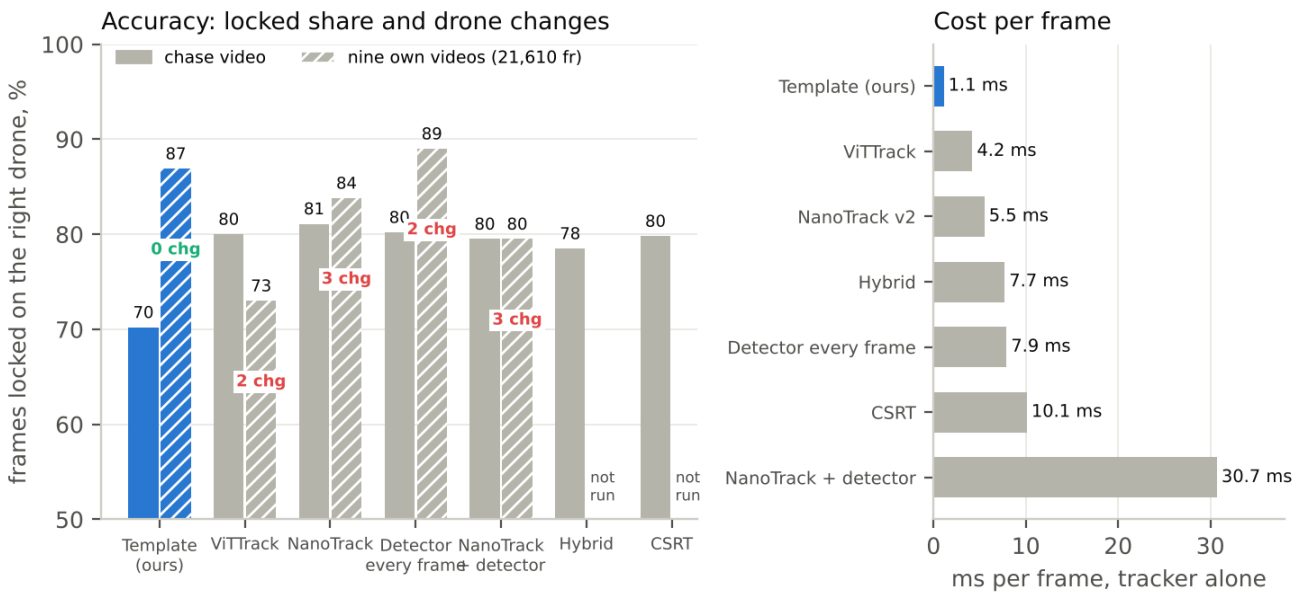


Figure 6. Trackers at a glance. Left: share of frames locked on the right drone (chase video and nine own videos), with the number of drone changes written on the bars where it is not zero. Right: cost per frame. The template tracker is the only one with zero changes on our videos, and the cheapest by far.

On the chase video every learned tracker beats the template by about ten points: they hold on through the fast aspect changes of a close drone that a 32 px template cannot represent. But on the nine-video set ViTTrack loses a quarter of the frames (it drifts off small, low-contrast drones and does not come back), NanoTrack spends 7.6 % of the time on the wrong object, and all of them produce two or three drone changes, which our rules forbid.

Running the detector on the crop every frame is the strongest in raw lock rate (89.0 %) but also changes drone twice, because a detector has no notion of *which* drone, and it costs seven times more. We also tried OSTrack, MixFormerV2, SAM 2 and XMem: too slow for the budget, or built for large objects.

Two cheap fixes to the template tracker, the most recent changes, recovered part of the gap on the chase video (70.2 % → 72.4 %) without touching the rest (exam 91.9 % → 92.5 %, nine videos unchanged, one fewer drone change): after a miss the velocity is measured from the last *good* position rather than the guessed one (VEL_FIX), and motion is accepted only near the prediction and never changes the velocity estimate (MOTION_STRICT). A "discard list" of other drones seen during the loss (never re-acquire towards them) helped on the chase video and hurt in races and swarms, so it is an option, off by default.

2.7 Multi-drone variant

A variant of the loop tracks every detected drone at once, each with its own number, its own per-frame tracker and its own appearance gallery, while the escort still follows only one. It uses the same detector, the same tracker and the same identity rules as the single-target loop; only the bookkeeping changes. We use it to count drones per frame and distinct identities on swarm videos (Section 4): a detector that fragments one drone into several, or invents new ones, shows up there before it shows up in the single-target metrics. The rules, with the constants in use:

- **Detection.** The detector runs on the whole frame every 0.1 s (every third frame at 30 fps), with the zoomed re-check; boxes larger than about 270 px at 1280 are discarded. Between detector passes each numbered drone is carried by its own tracker (template, motion, 0.5 s of coasting, Section 2.3).
- **Association.** Every live drone takes the nearest box inside its gate, max(18 px, 1.5 × its size), or 0.8 × its ROI half-side when its tracker is failing. Pairs are resolved nearest first, one box per drone, and a box that the drone's gallery judges *different* is skipped. A drone left without a box gets a second chance with a wide gate (4 × size + 60 px), again under the gallery's veto.
- **A new number.** A box left over with confidence ≥ 0.50 opens a new number if it reaches 0.80 in one pass or appears in three coherent passes (within max(30 px, 2 sizes)), provided it is not glued to a live drone (closer than 1.2 × size), no live drone's gallery recognises it as *same*, and it is not a lost drone coming back. At most eight numbers are live at once.
- **Retiring a number.** A drone is retired when the detector has not confirmed it for 1 s (2 s while its template still matches at ≥ 0.75; 0.6 s for a large drone that is alone), or when it leaves the frame.
- **Reserved numbers.** A lost drone keeps its number and its gallery for 10 s. It gets them back for a candidate that its gallery judges *same*, anywhere in the frame; for a candidate on its extrapolated trajectory (for 3 s, inside a zone that grows with time) that the gallery does not deny; or, when it is the only recent loss (under 5 s) and there is a single candidate. A *different* verdict never takes over a number.
- **Telling drones apart.** There is no classification by drone model: there is one class, "drone", and identity comes from appearance and trajectory. Each number keeps a gallery of up to ten samples, one every 15 frames and only right after a detector confirmation: a grayscale crop and a hue histogram of the drone's lights and coloured

parts (18 bins over saturated, bright pixels). A candidate scores 0.35 × shape (normalised cross-correlation at six scales) + 0.65 × colour (1 − Bhattacharyya distance); *same* needs ≥ 0.65 with shape ≥ 0.45 and colour ≥ 0.70; *different* is ≤ 0.30; anything in between is *unknown*, and so is a drone under 14 px or touching the border, or a candidate more than three times larger or smaller than the last sample. Without colour there is never a *same* or a *different*: two drones of the same model cannot be told apart by shape alone.

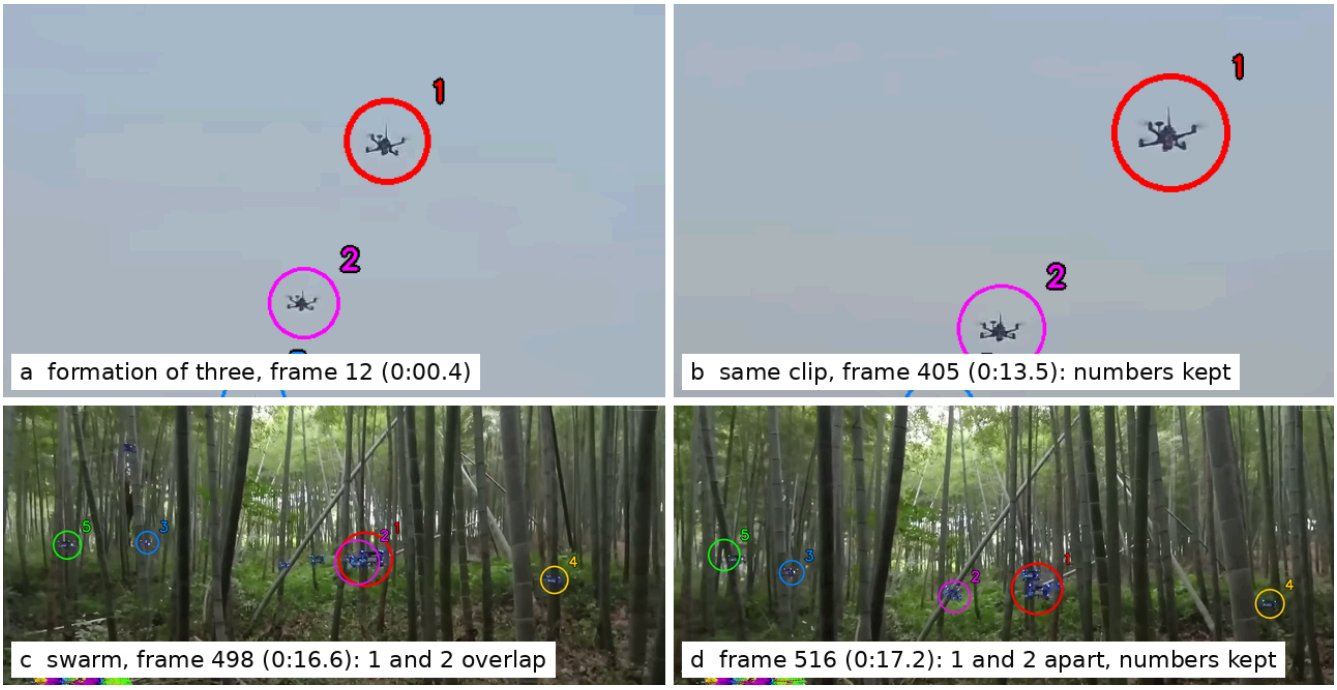


Figure 7. The multi-drone variant on two public clips. Top: a formation of three (vertical phone video, cropped); numbers 1, 2 and 3 are assigned within the first six frames and kept for the whole 15 s clip (all three numbered in 445 of 451 frames, no loss); the right panel is the closest approach, 85 px between drones 2 and 3, thirteen seconds later. Bottom: a swarm of identical drones in a bamboo forest; up to six numbered at once and eight numbers used in 34 s. Left, frame 498: five numbered while drones 1 and 2 overlap in the line of sight (centres 25 px apart). Right, frame 516: they separate with their numbers; drone 2 was lost during the overlap and recovered five frames later by trajectory. A corner of the clip's own overlay graphics shows at the bottom left.

The swarm clip also shows the limits honestly. Its drones are identical and colourless, so the gallery never returns *same* or *different*: continuity through the overlap rests on trajectory alone and cannot be verified from the video, the small drones at the back go unnumbered, and two large jumps earlier in the clip (numbers 4 and 5, by about 500 and 140 px) are probably drone changes that the single-target rules would have refused. A clean test of identity by colour needs drones with different lights, which in our material exist only in the race-drone sessions.

3. The detector: data and training

3.1 Training data

The adopted model, which we call **cv15**, trains on **34,854 images** from ten sources, five of them public (Table 4, Figure 8). Every colour source except the sky synthetics is present twice, once as is and once converted to grayscale (3-channel JPEG, labels unchanged).

Table 4. Training sources of cv15. Public sets are linked in the references; our own footage is described in Section 4.

Source	Colour	Grayscale copy	Content
DUT Anti-UAV, train split [1]	5,200	5,200	Public; mostly small drones against sky and urban backgrounds, filmed from the ground
Our own videos	3,712	3,712	Frames from our chases, formations, race-drone sessions and a garage, boxes checked by eye; test videos removed
Sky synthetics	3,000	—	Drone cut-outs shrunk to 6–18 px and pasted on sky-only frames (the only synthetic data that helped)
FPV chase set (Roboflow Universe) [4]	1,852	1,852	Public; first-person-view footage of racing quads chasing each other
NTUT Air-to-Air Drone dataset (Roboflow Universe) [5]	1,618	1,618	Public; drones filmed from another drone over buildings, grass and trees; every second frame
YouTube positives	1,577	1,577	Hand-reviewed frames from public videos: a consumer camera quad, FPV quads, an RC plane, a fixed-wing foam drone, a swarm of three
YouTube negatives	1,266	1,266	Frames with people, birds, floodlights, antennas and screens; no boxes
Race-drone crops	522	522	512×512 crops of LED race drones over vegetation and inside the garage
Auto-mined positives	166	166	Detections confirmed by the tracker and then reviewed by eye
Mined negatives	14	14	Frames containing our own earlier false positives, no boxes
Total	**18,927**	**15,927**	**34,854**

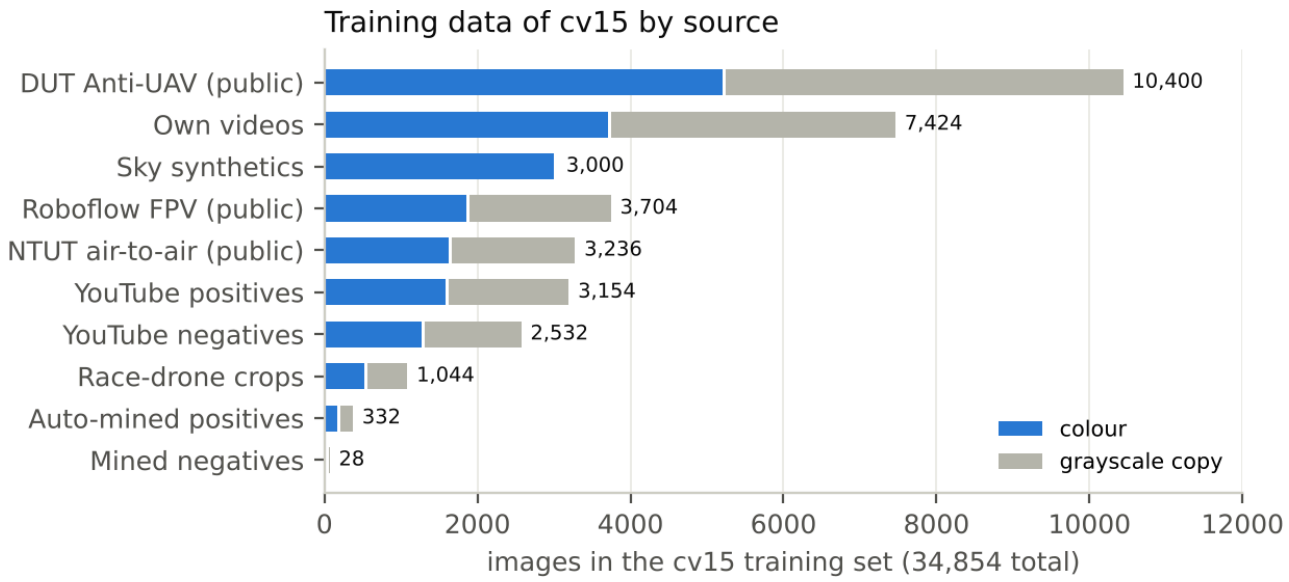


Figure 8. Training images per source. The grayscale twin doubles everything except the sky synthetics.

The rules we apply when adding data turned out to matter as much as the data itself:

- **Test videos never enter training.** The four exam videos and the reserved segments of four more (Section 4) are excluded by file lists. Our early models had frames of two exam videos in training; the models named `cv` (“clean”) fixed that, which is why `cv7` scores lower on the exam than `v7` trained on the same recipe.

- **Positives bring their own negatives.** When frames of a video are added, background frames of the same video are added too, and every false positive found during testing becomes a negative frame (never from the test videos).
- **Only checked boxes.** Automatically mined detections are added only after visual review on contact sheets.
- **Negatives must look like our scenes.** Generic negatives hurt (Section 6).
- **Variety over volume.** A new *kind* of footage moves the test score; more frames of a scene we already have does not, and does not even generalise to the other half of the same video.

Sets we tried and dropped: Det-Fly [2] and NPS-Drones [3] (used in intermediate models; they diluted our own footage and cost the 10 px drones, see Section 6), two bird datasets as negatives (worse), and a set of airborne-object clips that did not match our scenes.

3.2 Training recipe

cv15 is Ultralytics YOLO26-s [6] fine-tuned from the COCO checkpoint for **80 epochs** with the framework defaults; the only choices we made are the model size, the epoch count and the data (Table 5). One training takes 4.9 h on the desktop GPU of Section 9.1.

Table 5. Training configuration of cv15 (everything not listed is the Ultralytics 8.4 default).

Setting	Value	Why
Architecture	YOLO26-s: 120 layers, 9,465,567 parameters, 20.8 GFLOPs	YOLO26-n was too weak on small drones at 1280; larger variants do not fit the Pi/Hailo budget
Starting weights	COCO-pretrained `yolo26s.pt`	Training from scratch gave fewer false positives but a worse escort (model v13z)
Epochs	80	50, 60 and 70 all gave more false positives at equal recall (Section 5.3)
Image size (training)	640	The same weights are run at 800×1280 and 640×640 at inference
Batch	32	8.5 GB of GPU memory
Optimizer	`auto` (SGD/AdamW chosen by the framework), lr0 0.01, final lr 1 %, momentum 0.937, weight decay 5×10^{-4}, 3 warm-up epochs	Defaults
Augmentation	mosaic (off for the last 10 epochs), scale ±50 %, translate 10 %, horizontal flip 0.5, HSV jitter 0.015/0.7/0.4, random erasing 0.4; no mixup, no vertical flip	Defaults
Precision, seed	mixed precision, seed 0, deterministic	
Validation during training	DUT Anti-UAV val split (2,600 images, 2,620 drones)	Sanity check only
Software	Python 3.14, PyTorch 2.14.1 (CUDA 13.0), Ultralytics 8.4.173, TensorRT 11.3, OpenCV 4.14 (Section 9.1)	

The final DUT validation reads precision 0.969, recall 0.897, mAP50 0.925, mAP50-95 0.619 (Figure 12). We do not select models on these numbers: every model we trained lands between 0.92 and 0.93 mAP50 on DUT regardless of how it behaves in the escort, because DUT is easier than our footage.

3.3 Deployment

`best.pt` is exported to ONNX four times (640 dynamic batch 1–8 for crops; 800×1280 for the full frame; 1088×1920 and 1920×1088 for phone footage) and compiled to TensorRT FP16 engines. Measured on the same 16 GB desktop GPU, otherwise idle, the full-frame search with its zoomed re-check has a median of 4.5 ms per frame (95th percentile 11 ms), the tracker 0.55 ms per frame, and the 640 px confirmation crop 6.2 ms in its own thread every 0.1 s; while locked, the loop therefore costs about 2.5 ms per frame on average, and about 5 ms while searching. Engines are specific to the GPU and TensorRT version and are rebuilt on each machine.

4. Evaluation protocol

We do not trust mAP on a public split. Every model is scored on our own videos, with the detector alone and with the detector inside the escort, because a model that looks better alone can make the tracker jump to the wrong drone. Table 6 describes the videos; they are internal recordings and public videos we cannot redistribute, so we describe them.

Table 6. Test material. "Exam" videos are never used for training in any way; "reserved" segments are excluded from training while the rest of that video may be used.

Label	Content	Role
chase2	2 min 30 s, 4,489 frames at 30 fps. FPV footage of a race quad chasing another through and around an abandoned concrete building on farmland; target 13–140 px, often against walls, ground or vegetation, fast pans, two drone-sized distractors (a red car, a white satellite dish). The hardest sequence.	Exam
video4	Handheld from the ground, a quad over countryside against bright sky and tree lines; drone mostly small.	Exam
yt8_a, yt8_b	Public video, a city skyline with cranes and a fixed mast light; three drones and birds (a), one drone (b).	Exam
video2, video3, video5, video6 (reserved segments)	A garage session with LED race drones; a hexacopter over trees; a distant drone in blue sky with insects near the lens; race drones at dusk over pines, filmed vertically.	Outdoor exam
“Nine videos”	video1, video4, video5, chase2, a formation flight and five short race clips; 21,610 frames with a drone and a hand-made position reference.	Escort set
YouTube chases	Public FPV chases yt1–yt3 (10,486 frames) and a wider set of 32,623 frames; split by distance (far / medium / near).	Escort set
yt4, yt7, yt5	Three drones and a bird; a swarm of ten; nine drones.	Multi-drone counts
NPS-Drones clips 41–50	Fixed-wing drones, never trained on.	Mandatory check

1. Clean exam (detector alone). The four exam videos sampled every 10 frames: 1,461 frames with **2,169 drone instances**. Every candidate box of every model was printed on contact sheets and reviewed by hand as *drone, not a drone* or *doubtful* (ignored). Two numbers come out: **drones seen (%)** and **false boxes per 100 frames**. The threshold is chosen per model; cv15 uses 0.25 plus the zoomed re-check.

2. Outdoor exam (detector alone). The reserved segments: 486 frames, 378 drones, same procedure. Harder footage: 10 px drone in blue sky, hexacopter over trees, a blurred segment inside the abandoned building.

3. Escort runs. The full loop is run on each video against a hand-made reference of the drone position. We report the share of frames with the drone where the track is **on it**

("locked"), **on something else** ("wrong"), or **absent** ("gap"), plus the number of **drone changes**, which must be zero, and the number of gaps longer than 0.25 s. The escort is reported on three sets (exam videos, nine own videos, YouTube).

4. Multi-drone counts. Drones per frame and distinct identities on the swarm videos.

Everything is scripted: after each training one script exports the engines and runs the whole battery (about 40 minutes), so every model in Section 5 was measured the same way. Before a model is adopted it must also be watched on chase2, video4 and the NPS clips.

5. Results

5.1 Model history

Table 7 lists every model that reached the clean exam, and Figure 9 shows the trade-off that decided between them. Models marked * had frames of two exam videos in training, so their exam scores are optimistic.

Table 7. Model history, in chronological order. Clean exam = detector alone (seen % / false boxes per 100 frames). Escort exam = locked share on the same four videos. Nine videos = locked share (drone changes).

Model	Recipe	Epochs / start	Clean exam	Escort exam	Nine videos
v7*	DUT + own videos + sky synthetics + mined negatives	chained: 80 / COCO, then 40, then 30	88.0 % / 19.2	89.0 %	87.8 % (1)
v9–v11	v7 + NPS-Drones + Det-Fly + race crops, own videos re-weighted ×2–3	20 / v7	not on exam		lost the 10 px drones; dropped
v13*	v7 + hand-reviewed YouTube	20 / v7	92.9 % / 6.7		87.0 % (5)
v13z*	same data as v13	80 / COCO	92.8 % / 5.9		86.0 % (2)
v13ze	v13z fine-tuned on own videos	15 / v13z			85.3 % (1), forgot swarms and YouTube
v13ze2*	v13z fine-tuned, backbone frozen	15 / v13z	94.2 % / 7.5	89.3 %	
cv7	v7 recipe, exam videos removed	80 / COCO	86.3 % / 10.1	87.0 %	
cv13	v13 recipe, exam videos removed	80 / COCO	88.6 % / 13.6	86.2 %	
cve2	cv13 fine-tuned, backbone frozen	15 / cv13	88.4 % / 17.9	87.0 %	
cv14	cv13 + synthetic drones over ground	80 / COCO	88.5 % / 16.6		
cv15	**cv13 + FPV chase set + NTUT air-to-air**	**80 / COCO**	**93.3 % / 3.6**	**93.4 %**	**87.8 % (0)**
cv16	cv15 + 2,000 bird negatives	stopped at epoch 10			superseded by cv17
cv17	cv15 + bird + 4,606 COCO negatives	80 / COCO	84.8 % / 8.4		85.8 % (2)
cv17b	cv15 recipe	**50** / COCO	95.3 % / 11.4		88.2 % (2)
cv18	cv15 + 112 hard positives + 10 hard negatives, each ×5	80 / COCO	94.0 % / 10.1	92.9 %	87.6 % (1)

Two readings of the table. First, a higher “seen” figure is good only if the false rate stays low: cv17b and cv18 see one or two points more than cv15 but with three times the false boxes, and in the escort that becomes drone changes (2 and 1 versus 0). Second, the escort numbers move much less than the detector numbers (86–89 % on the nine videos for every model) because the tracker carries most frames; the detector matters at acquisition and after a loss, which is exactly where chase2 (50.5 % for cv13 → 75.0 % for cv15) separates the models.

On the outdoor exam (378 drones) cv15 scores 74.1 % seen with 0.4 false boxes per 100 frames; cv17 64.0 % / 0.0, cv17b 77.2 % / 1.9, cv18 73.0 % / 0.6.

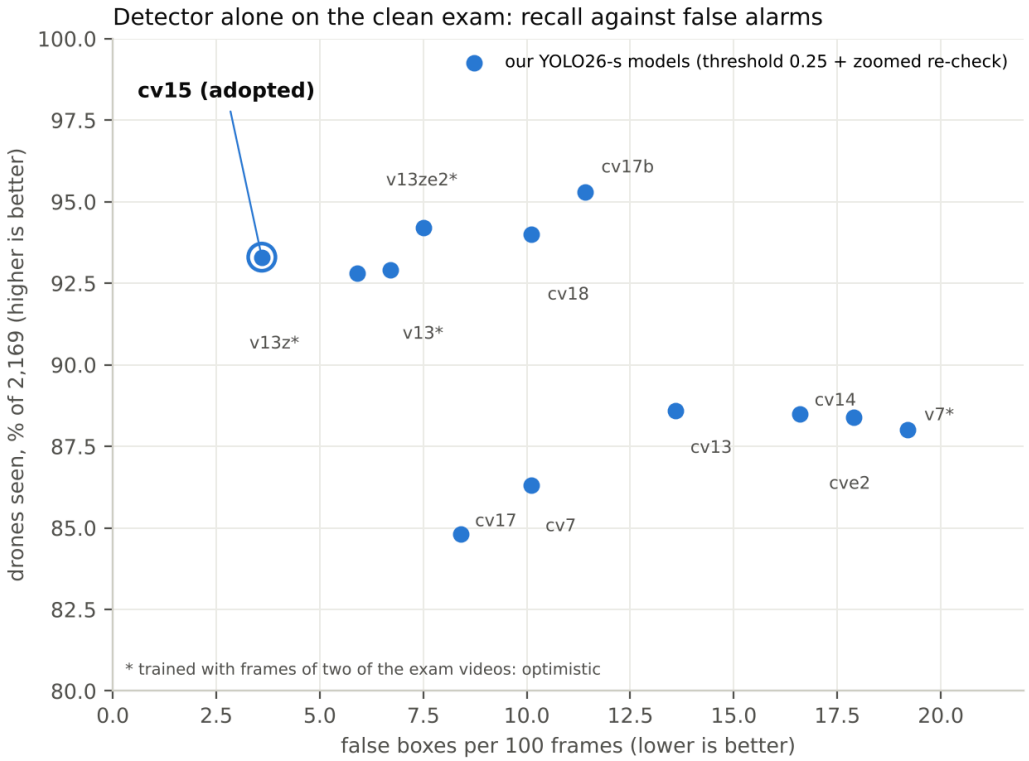


Figure 9. Detector alone on the clean exam. Each point is one of our models at its operating threshold. Up and to the left is better. cv15 is the only model under 4 false boxes per 100 frames above 93 % recall.

5.2 Escort results

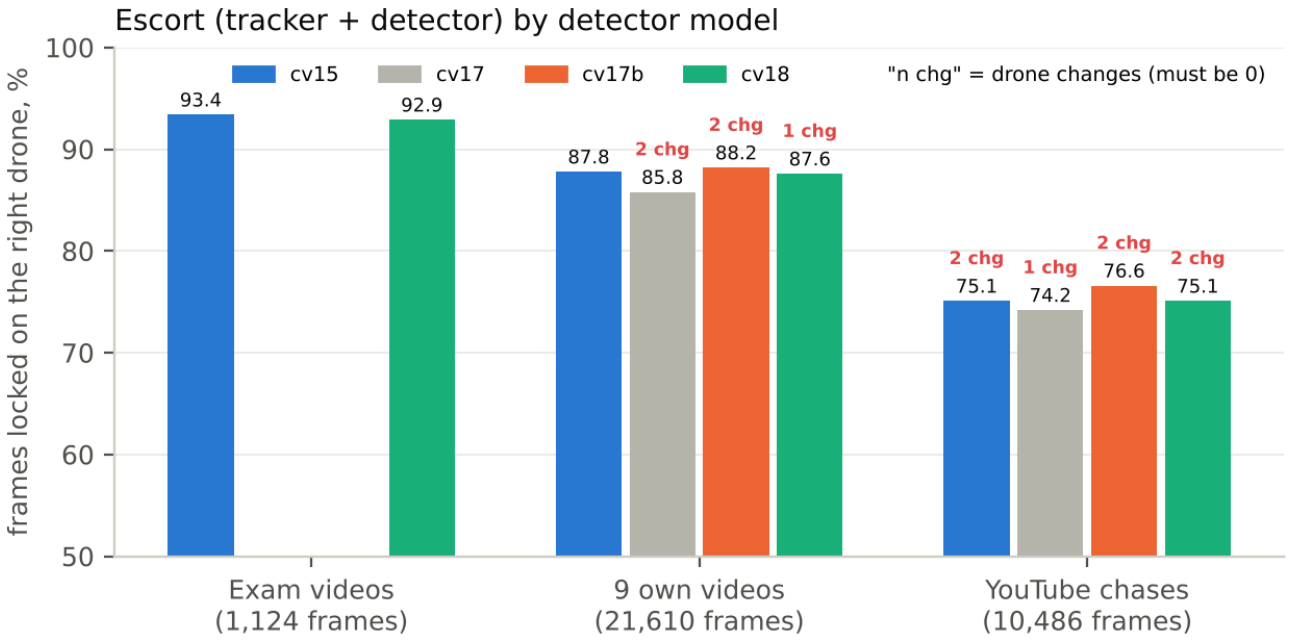


Figure 10. Locked share of the escort loop by detector model, same tracker. The differences are small; what separates the models is the number of drone changes (red), which only cv15 keeps at zero on the own-video set.

With cv15 and the default tracker (template, VEL_FIX, MOTION_STRICT) the current figures are: exam videos 92.5 % locked, 0.7 % wrong, no drone change; nine own videos 86.5 % locked, 2.3 % wrong, one change; YouTube set 87.1 % locked, 4.6 % wrong; chase2 72.4 %. On the YouTube chases the lock rate falls with distance: 78.9 % near, 75.5 % medium, 35.0 % far (drones of about 10 px with the camera panning).

5.3 Why 80 epochs

cv17b is the cv15 recipe trained for 50 epochs. It sees more drones at the default threshold (95.3 % against 93.3 %) but with 11.4 false boxes per 100 frames. Sweeping the confidence threshold (Figure 11) shows that at any given false rate the two models see about the same share of drones, and that cv17b needs a threshold of 0.65 to come down to cv15’s false rate, where it sees 74.8 %. Sixty and seventy epochs gave the same picture. The extra epochs buy precision, not recall, and precision is what keeps the tracker from jumping.

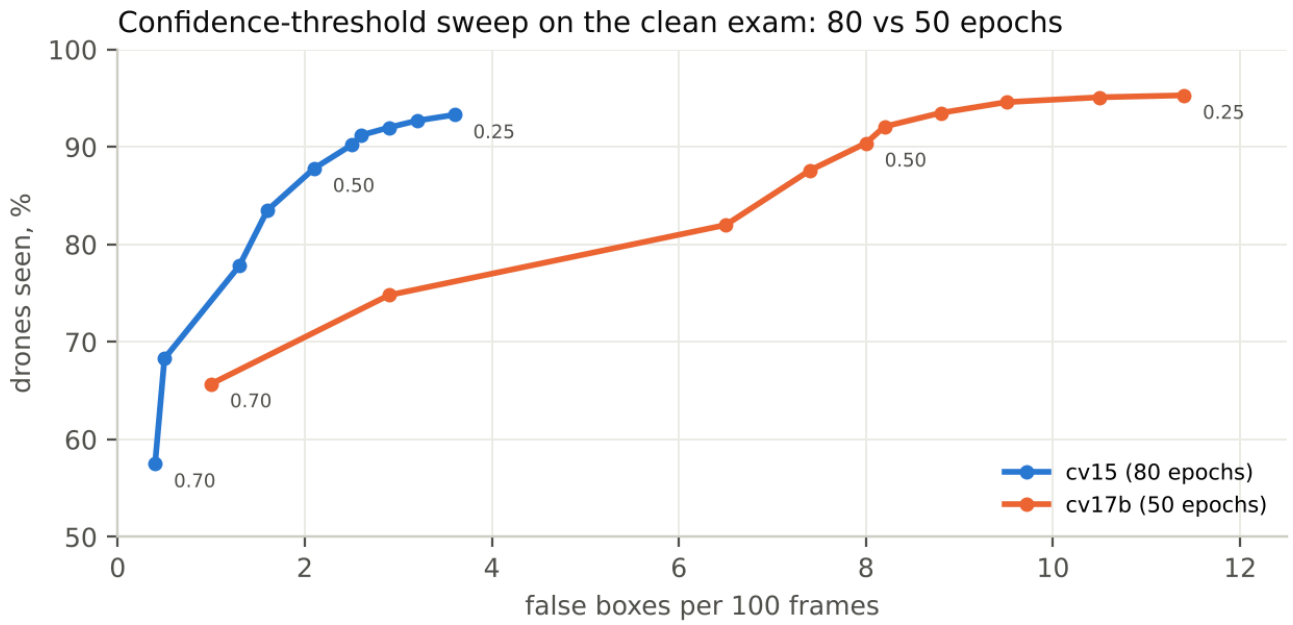


Figure 11. Confidence-threshold sweep on the clean exam. Labels mark thresholds 0.25, 0.50 and 0.70.

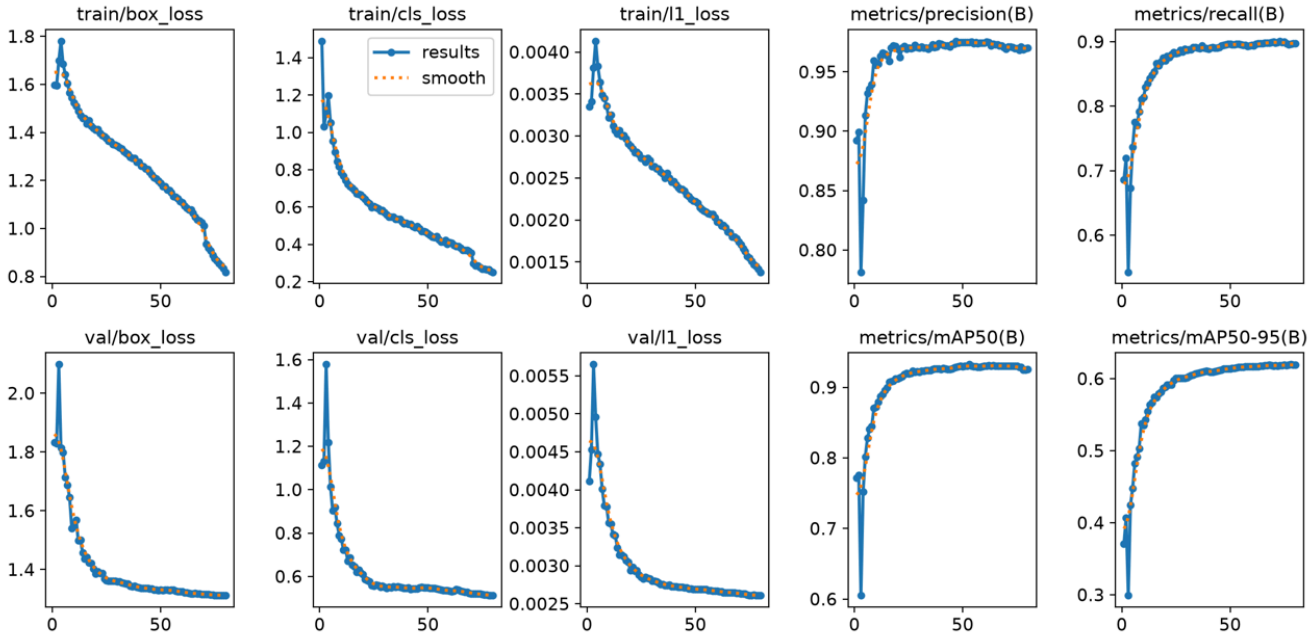


Figure 12. Training curves of cv15. Losses are still falling slowly at epoch 80 while the DUT metrics have been flat since about epoch 40; the extra epochs matter on our videos, not on DUT.

5.4 What the detector sees and misses



Figure 13. NTUT air-to-air samples, one of the two public sets that made the difference between cv13 and cv15: drones seen from another drone against buildings, grass and trees, which DUT almost never shows.

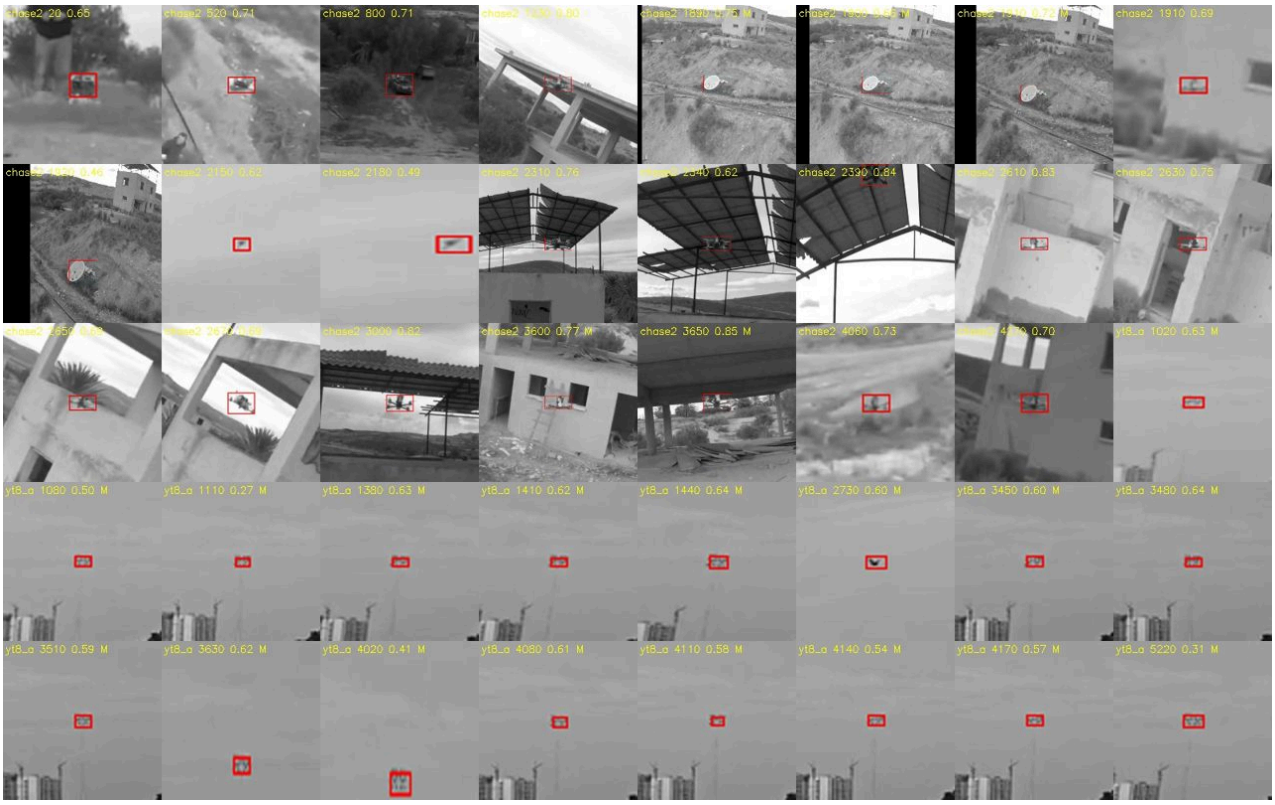


Figure 14. False positives of cv15 on the clean exam (contact sheet from the hand review, grayscale run). Rows 1–3, chase2: a satellite dish, a red car, window openings of the building, a person. Rows 4–5, yt8_a: a fixed light on a mast and birds. Future negatives should look like these.



Figure 15. A case cv15 still misses: the yellow race drone of chase2 at 18–25 px over bare ground and walls (frames 555–615). The drone is visible to a person on every frame.

6. What worked and what did not

Worked

- Public air-to-air and FPV sets (Table 4): the single biggest step, cv13 88.6 % / 13.6 → cv15 93.3 % / 3.6, chase2 50.5 % → 75.0 %.
- Hand-reviewed YouTube positives *and* negatives (v12/v13): +4 points seen and a third of the false boxes.
- Removing the test videos from training: about 2 points lower on paper, numbers we can trust.
- 80 epochs from the COCO checkpoint, framework defaults, one training run per model. No chained fine-tunes.
- Colour: tracking is clearly better in colour than in grayscale (in grayscale the template locks onto terrain), which drove the change of sensor.
- The zoomed re-check (Section 2.1) and the “better lost than wrong” rules (Sections 2.4–2.5).

Did not work

Table 8. Approaches that were tried and dropped, with the measurement that dropped them.

Attempt	Result	Why we think it failed
Synthetic drones pasted over ground seen from above (cv14)	88.5 % / 16.6 false, worse than its base cv13	Paste artefacts; the model learned the cut-out. Sky-only synthetics are the exception and stayed
Bird photos as negatives (cv16) and birds + 4,606 COCO images as negatives (cv17)	cv17: 84.8 % / 8.4; outdoor exam 64.0 %	Generic negatives pulled the model away from our scenes: 8 points of recall for a modest gain in false rate

Attempt	Result	Why we think it failed
50 instead of 80 epochs (cv17b); also 60 and 70	95.3 % / 11.4; two drone changes on the own videos	Same recall at equal false rate; the extra false boxes make the tracker jump (Figure 11)
Hard-mined positives and negatives from our own failures, repeated ×5 (cv18)	94.0 % / 10.1; escort equal or worse (chase2 69.6 % vs 72.5 % locked)	112 positives from our failure cases taught the model our videos, not the general case; false boxes tripled
Fine-tuning a general model on our videos for 15 epochs (v13ze)	chase2 69 %, YouTube 71.8 %, swarm 0.96 drones per frame	Catastrophic forgetting of swarms and YouTube drones
Training from scratch instead of COCO (v13z)	92.8 % / 5.9 but a worse escort	Cleanest detector of its generation but sees less
Re-weighting our videos ×2–3 against NPS-Drones and Det-Fly (v10, v11)	Lost the 10 px drones of one video	DUT, the source of tiny drones, fell to 13 % of the mix
YOLO26-n	Dropped early	Too weak on small drones at 1280
A second classifier after the detector to reject false positives (a 64 px YOLO26 classifier)	Rejected real drones along with the false ones	Small drones carry too little appearance to classify reliably
Stationary-object filter (discard what has not moved for 3–6 s)	88.7–91.3 % seen, same false rate	Removes hovering drones, keeps the mast light
Running the detector at 1920 instead of 1280	Escort exam 92.2 % vs 91.5 %, nine videos 86.1 % vs 87.3 %	Sees one more distant drone on chase2, slower, no better overall; the camera is 1280 anyway
Wider AI review while the tracker is unsure	Escort exam 92.5 % → 88.3 %	More detector calls, more chances to lock onto the wrong thing
Learned trackers in place of the template (Table 2)	Better on chase2, worse on the rest, drone changes	No notion of which drone; drift on small low-contrast targets
Classical detection without a network (frame differencing, propeller signature)	10–20× faster but far more losses (chase2 42 % locked)	The propeller signature disappears during a chase

7. Limitations and open problems

- **Small drones over ground, roofs and tree tops seen from above** are the main remaining miss (Figure 15). Our data has few such frames; a photorealistic simulator confirmed that cv15 barely sees a 56×32 px FPV drone over pine canopy from above. Real footage of that kind is the next data we want; we have decided not to train on simulator frames until it is measured against real data.
- **Resolution.** At 1280 px a drone at 13 px over moving vegetation is at the limit of what the detector and the template can use; at 1920 the detector sees it but everything else gets slower and no better. A P2 (stride-4) detection head is the untested idea.
- **Remaining false positives** on fixed lights, birds, red cars, white dishes and people (Figure 14). Mining negatives from our own failures made things worse (cv18); selective, small sets of outdoor negatives are the next attempt.
- **Fast pans** blur everything; the tracker waits, the detector sees nothing, and recovery depends on the drone being found again when the pan stops.
- **Several identical drones** (races) remain the hardest case for identity: shape is the same, and only the colour of lights separates them.
- **Edge deployment** on the Raspberry Pi 5 + Hailo is measured but not integrated: about 33 ms per tracked frame and 48 ms per searched frame on the Pi with an earlier model.

8. Reproducing the detector

The following recipe reproduces cv15 from public data plus footage of your own. With the same data and seed the result should match to within a few tenths of a point (GPU kernels are not bit-exact). Our own material (the videos and their annotated frames) is not published; the recipe says what it contributed so that equivalent footage can be substituted.

1. **Environment.** A Linux or Windows machine with an NVIDIA GPU of 16 GB (ours is described in Section 9.1). Python 3.12 or newer (we used 3.14), PyTorch 2.14 with CUDA 13, Ultralytics 8.4.173, OpenCV 4.14, ONNX 1.23 and, for deployment only, TensorRT 11.3.
2. **Public data.** Download DUT Anti-UAV [1] (detection part: 5,200 train, 2,600 val, 2,200 test images) and convert its XML annotations to YOLO format (one class, id 0, normalised `cx cy w h`). Download the NTUT Air-to-Air Drone dataset [5] and the FPV chase dataset [4] from Roboflow Universe in YOLO format; we used every second NTUT frame (1,618 of 3,236) and all 1,852 FPV images.
3. **Your own footage (substitutes for our 3,712 + 522 + 166 frames).** Record or collect videos of the drones you will escort in the scenes you fly: chases, formations, drones over ground and vegetation, drones with LEDs. Sample about one frame per second, label the drones, and add background frames of the same videos without boxes. Review every box by eye. Keep one or two whole videos and a few segments aside as test material and never train on them.
4. **YouTube-style positives and negatives (substitutes for our 1,577 + 1,266 frames).** Public videos of drones of several types (consumer quads, FPV quads, fixed wings, small swarms): sample frames, label and review. Collect negative frames of the usual distractors: people, birds, floodlights, antennas, screens, cars.
5. **Sky synthetics (3,000 images, optional but used).** Cut drones out of labelled frames, shrink them to 6–18 px and paste them onto frames that contain only sky, writing the box. Do not paste onto ground, trees or buildings: that variant (cv14) hurt.
6. **Grayscale twins.** For every folder of colour images except the synthetics, make a copy converted to grayscale (saved as 3-channel JPEG) with the same labels. Skip this if your camera will always be colour; we have not measured training without the twins.
7. **Data configuration.** One Ultralytics data configuration listing every folder (or image-list file) as training data, the DUT validation split as validation data, and a single class named *drone*. Our list has 21 entries: 17 for the clean base (DUT, own videos, auto-mined, synthetics, race crops, YouTube positives and negatives, each with its grayscale twin) plus the four public FPV/NTUT folders.
8. **Train.** With the Ultralytics API, from the COCO-pretrained YOLO26-s checkpoint: 80 epochs, image size 640, batch 32, initial learning rate 0.01, one GPU, four data-loader workers; everything else an Ultralytics default. Expect about 5 hours on a mid-range 16 GB GPU and 8.5 GB of GPU memory (Section 9.2 gives the measured times of every run). The run produces the per-epoch log, the plots of Figure 12 and the best weights; the final DUT validation should read roughly P 0.97, R 0.90, mAP50 0.925.
9. **Export.** From the best weights, export ONNX at 640 (dynamic batch) and at the camera resolution, 800×1280, in FP16 with a static shape and the graph simplified; then build TensorRT engines on the target machine. Engines are tied to GPU and TensorRT version.

10. **Measure before adopting.** Build a clean exam of your own (Section 4): sample test videos every 10 frames, run the detector, print every candidate box on contact sheets, review by hand, and report drones seen and false boxes per 100 frames. Then run the full escort loop and count drone changes. Our bar: ≥ 93 % seen at ≤ 4 false per 100 on the exam, and zero drone changes on the escort set.
11. **Run.** Full-frame search at 800×1280 with threshold 0.25 and the zoomed re-check (Figure 2); the same weights at 640 for crops; the tracker and rules of Section 2. The constants of Figures 1–5 are the ones in use.

To train a successor, add new source folders to the YAML, keep 80 epochs from the COCO checkpoint, and never chain a fine-tune on top of a previous model: every attempt at that (v13ze, cve2) forgot the general case.

9. Hardware, compute and the simulator

9.1 Hardware and software

Everything in this report, from training to the simulator, ran on one desktop workstation:

- **CPU**: Intel Core Ultra 7 265F, 20 cores (8 performance + 12 efficient), no integrated graphics.
- **GPU**: NVIDIA GeForce RTX 5070 Ti with 16 GB of GDDR7 memory. It is shared by training, the evaluation battery and the simulator, which is why the timings of Section 3.3 were measured with the GPU otherwise idle.
- **Operating system and software**: Windows 11; Python 3.14 in a conda environment; PyTorch 2.14.1 with CUDA 13.0; Ultralytics 8.4.173; TensorRT 11.3 for the deployed engines; ONNX Runtime GPU 1.30; OpenCV 4.14 with the contrib modules. A second environment with the same stack hosts the simulator client.
- **Target camera**: an OV9782 colour global-shutter sensor (1280×800) behind an M12 lens of about 60° horizontal field of view; part of our own footage comes from its monochrome twin, the OV9281. The detector runs at this camera's resolution (800×1280) in every measurement of this report.

9.2 Compute used

Ultralytics writes the elapsed time of every epoch to `results.csv`, so training time is known exactly for all 37 runs of the project (Table 9, Figure 16). **cv15 took 4.9 h** for its 80 epochs over 34,854 images, about 3.7 minutes per epoch at batch 32 and 640 px. **All training together took 67.0 GPU-hours**: 47.8 h for the seventeen YOLO26-s models of Table 7, 11.9 h for eight early YOLO26-s experiments that never reached the exam, 7.3 h for nine YOLO26-n runs from the first days (the size we discarded), and a few minutes for three small classifiers we tried and dropped. Per-run times are not strictly comparable: a few runs shared the GPU with benchmarks or with the simulator (cv17, 7.2 h, also had the largest training set, 41,460 images), and cv16 was stopped after ten epochs.

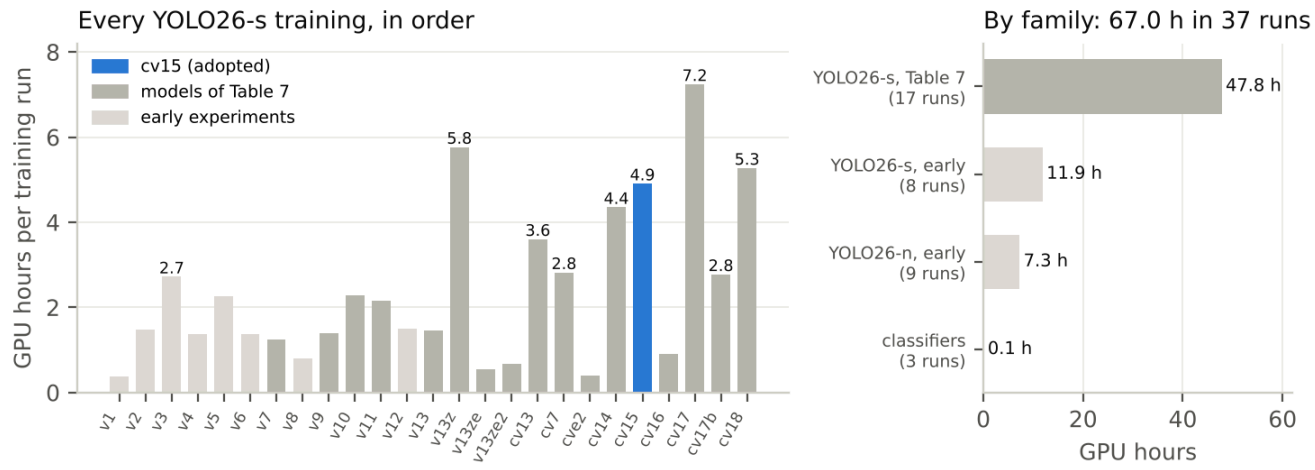


Figure 16. Training cost. Left: every YOLO26-s run in the order it was trained (early experiments in light grey, the models of Table 7 in dark grey, cv15 in blue). Right: the 67.0 GPU-hours by family.

Table 9. GPU hours of training, measured from the per-epoch times that Ultralytics records. "Early" runs are the ones that never reached the clean exam.

Family	Runs	GPU hours	Notes
YOLO26-s models of Table 7	17	47.8	from 0.4 h (15-epoch fine-tunes) to 7.2 h (cv17)
YOLO26-s, early experiments	8	11.9	9–80 epochs, one run at 1280 px
YOLO26-n, early experiments	9	7.3	the smaller model size, discarded
Small classifiers	3	0.1	drone-type and verifier experiments, dropped
All training	**37**	**67.0**	
cv15 alone	1	4.9	80 epochs, 34,854 images

GPU time outside training was not logged, so the rest is an estimate from what each step costs: the scripted evaluation battery takes about 40 minutes per model and ran for some fifteen models (about 10 h); the tracker comparisons, escort runs, ONNX and TensorRT exports and the timing benchmarks add roughly 10–15 h; and the simulator of Section 9.3, which renders at 15–30 frames per second, ran for roughly 15–20 h over its lens sweeps and closed-loop flights. In all, the project has used on the order of 100–110 GPU-hours, two thirds of them training.

9.3 The simulator

Real footage cannot tell us where the target really was, so the closed loop (detect, lock, follow, move the escort) is tested in a simulator that knows the true position of both drones. It is a test bench for the loop and for flight behaviour, not a substitute for the exam of Section 4: a rendered drone is never quite a real one, and simulator images are never used for training (synthetic drones already failed to help, Section 6).

What it is built from. Unreal Engine 5.8.4 with Cosys-AirSim 3.5.0 (the maintained fork of Microsoft AirSim for Unreal 5, which provides the quadcopter physics, the sensors and a Python client) and Cesium for Unreal 2.29.1 streaming Google Photorealistic 3D Tiles. The world is therefore a real place: a stretch of the Alicante coast with its dunes, pine woods, houses and sea, lit by a sun computed for the real local time of day. A first phase on AirSim 1.8.1 with a pre-built environment validated the circuit before the move to Unreal 5 and real terrain.

What flies in it. Two quadcopters. The target is a 3D model of an FPV racing quad that flies scripted trajectories (a forest run, a run over animals and an inspection run, each starting 30 m from the escort). The escort runs the same escort code as the real system on the frames of its simulated camera, which it receives as arrays, and turns the tracker's box into velocity, altitude and yaw commands for the simulated flight controller; the true position of the target is used only to score each run (locked share, distance, losses). The camera is made to match the real one: 1280×720, 60° horizontal field of view, motion blur, auto-exposure, sensor noise and compression, no propellers in view, on a one-axis pitch gimbal that is stabilised to the horizon and pointed at the target by the flight module (limits −90° to +30°). A one-point range finder along the camera axis (20 m) is logged for the distance experiments. The simulator uses about 6.8 GB of GPU memory, so it cannot run beside a training on the 16 GB card, and a closed-loop flight costs about four real minutes per ten simulated seconds, because every frame goes through rendering, the detector, the tracker and the physics in lockstep.

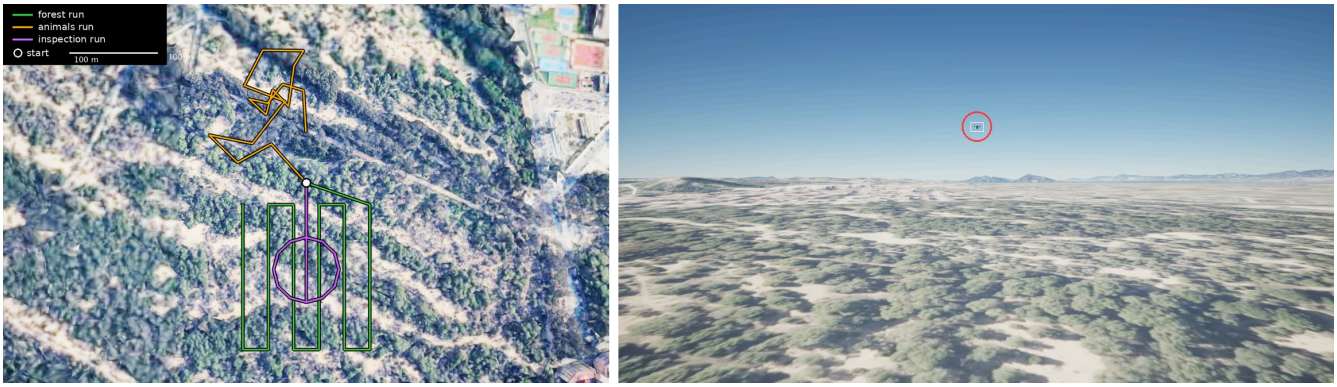


Figure 17. The simulator. Left: the three target trajectories over the real terrain (forest run in green, animals run in orange, inspection run in purple; the white dot is the start). Right: what the escort camera sees from 30 m above the ground with the FPV 20 m ahead (red circle): a few pixels over pine canopy, which cv15 does not detect in this frame.

What it has shown so far. (1) A lens sweep at 5–100 m: of a 60° lens, a 90° lens and a fisheye, the 60° lens is the only one with which cv15 still sees the FPV at 50 m, which fixed the lens choice for the real camera. (2) The closed loop with the simulated camera keeps the target locked for only 2.1 % (forest), 3.3 % (animals) and 0.1 % (inspection) of the flight, whereas piloting from the target's true position keeps it for 99.9 %: the weak link is between what the camera sees and the flight commands, not the flight controller. The gimbal oscillates in the first seconds (five reversals between +30° and −3° in ten seconds), after a loss the escort turns in place while the target flies away (nine of ten unlocked frames are at 55–200 m), and beyond 30 m over pine canopy the FPV is 6–10 px and cv15 does not see it. (3) The next steps are a slower gimbal, distance calibrated from the range finder and recovery by flying toward where the target will be. These findings feed the open problems of Section 7.

References

1. J. Zhao, J. Zhang, D. Li, D. Wang, "Vision-based Anti-UAV Detection and Tracking", IEEE Transactions on Intelligent Transportation Systems, 2022. Dataset and code: https://github.com/wangdongdut/DUT-Anti-UAV (arXiv:2205.10851).
2. Y. Zheng, Z. Chen, D. Lv, Z. Li, Z. Lan, S. Zhao, "Air-to-Air Visual Detection of Micro-UAVs: An Experimental Evaluation of Deep Learning", IEEE Robotics and Automation

Letters, 2021. Det-Fly dataset: https://github.com/Jake-WU/Det-Fly.

3. J. Li, D. H. Ye, T. Chung, M. Kolsch, J. Wachs, C. Bouman, “Multi-target detection and tracking from a single camera in Unmanned Aerial Vehicles (UAVs)”, IROS 2016 (NPS-Drones dataset).
4. FPV chase dataset, Roboflow Universe, 1,852 images, YOLO export (an unnamed community project; search Roboflow Universe for FPV drone chase datasets of similar size and content).
5. “NTUT Air-to-Air Drone dataset”, Roboflow Universe, 3,236 images, YOLO export: https://universe.roboflow.com (search by title).
6. Ultralytics YOLO26, Ultralytics 8.4: https://docs.ultralytics.com and https://github.com/ultralytics/ultralytics.
7. OpenCV Zoo trackers: NanoTrack v2 and ViTTrack: https://github.com/opencv/opencv_zoo; CSRT: A. Lukežič et al., “Discriminative Correlation Filter with Channel and Spatial Reliability”, CVPR 2017.